

2. Lập trình Javascript

»» Array

- **Array (Mảng)**: cho phép lưu nhiều giá trị phần tử (có thể không cùng kiểu dữ liệu) trong cùng 1 biến.

```
var beer1 = "Heineken";  
var beer2 = "Tiger";  
var beer3 = "Sapporo";
```



```
var beers = ["Heineken", "Tiger",  
             "Sapporo"];  
  
var beers = new Array("Heineken",  
                       "Tiger", "Sapporo");
```

Kiểm tra kiểu dữ liệu:

```
console.log(typeof beers); // object  
console.log(typeof {}); // object  
console.log(Array.isArray(beers)); // true  
console.log(beers instanceof Array); // true  
console.log(Array.isArray({})); // false  
console.log({} instanceof Array); // false
```

2. Lập trình Javascript

»» Array

- **toString()**: chuyển array thành chuỗi, các phần tử cách nhau bởi dấu “,”

```
const beers = ["Heineken", "Tiger",  
"Sapporo"];  
console.log(beers.toString());  
//--> Heineken,Tiger,Sapporo
```

- **join()**: nối các phần tử của array thành chuỗi, các phần tử cách nhau bởi chuỗi đối số của hàm.

```
const beers = ["Heineken", "Tiger",  
"Sapporo"];  
console.log(beers.join(" - "));  
//--> Heineken - Tiger - Sapporo
```

2. Lập trình Javascript

»» Array

- **pop()**: xóa phần tử cuối của array và trả về phần tử đã xóa.

```
const beers = ["Heineken", "Tiger", "Sapporo"];  
console.log(beers.pop()); // Sapporo  
console.log(beers); // ['Heineken', 'Tiger']
```

- **shift()**: xóa phần tử đầu của array và trả về phần tử đã xóa.

```
const beers = ["Heineken", "Tiger", "Sapporo"];  
console.log(beers.shift()); // Heineken  
console.log(beers); // ['Tiger', 'Sapporo']
```

2. Lập trình Javascript

»» Array

- **push()**: thêm các phần tử vào cuối array và trả về số lượng phần tử của array.

```
const beers = ["Heineken", "Tiger", "Sapporo"];  
console.log(beers.push("Corona Extra")); // 4  
console.log(beers);  
// ['Heineken', 'Tiger', 'Sapporo', 'Corona Extra']
```

- **unshift()**: thêm các phần tử vào đầu array và trả về số lượng phần tử của array.

```
const beers = ["Heineken", "Tiger", "Sapporo"];  
console.log(beers.unshift("Corona Extra")); // 4  
console.log(beers);  
// ['Corona Extra', 'Heineken', 'Tiger', 'Sapporo']
```

2. Lập trình Javascript

»» Array

- **splice(start, deleteCount, ...items):** xóa, thêm phần tử tại vị trí bất kỳ trong array.

```
const beers = ["Heineken", "Tiger", "Sapporo"];  
console.log(beers.splice(1, 1)); // "Tiger"  
console.log(beers); // ['Heineken', 'Sapporo']
```

```
const beers = ["Heineken", "Tiger", "Sapporo"];  
console.log(beers.splice(1, 1, "Corona")); // "Tiger"  
console.log(beers); // ['Heineken', 'Corona', 'Sapporo']
```

```
const beers = ["Heineken", "Tiger", "Sapporo"];  
console.log(beers.splice(1, 0, "Corona", "Strongbow")); // []  
console.log(beers);  
// ['Heineken', 'Corona', 'Strongbow', 'Tiger', 'Sapporo']
```

2. Lập trình Javascript

»» Array

- **concat()**: nối các array.

```
const beerList1 = ["Heineken", "Tiger", "Sapporo"];
const beerList2 = ["Corona", "Strongbow"];
console.log(beerList1.concat(beerList2));
// ['Heineken', 'Tiger', 'Sapporo', 'Corona', 'Strongbow']
```

- **slice(start, end)**: cắt array từ vị trí start đến < end.

```
const beerList = ["Heineken", "Tiger", "Sapporo"];
console.log(beerList.slice(1, 2)); // ['Tiger']
console.log(beerList.slice(-2, -1)); // ['Tiger']
console.log(beerList.slice(-1)); // // ['Sapporo']
```

2. Lập trình Javascript

»» Array

- `sort()`, `reverse()`: sắp xếp mảng.

```
const beerList = ["Heineken", "Tiger", "Sapporo"];
beerList.sort();
console.log(beerList); //['Heineken', 'Sapporo', 'Tiger']
beerList.reverse();
console.log(beerList); //['Tiger', 'Sapporo', 'Heineken']
```

```
const marks = [4, 2, 10, 8, 15];
marks.sort();
console.log(marks); // [10, 15, 2, 4, 8]
```

```
const marks = [4, 2, 10, 8, 15];
marks.sort(function(a, b) {
    return a - b;
});
console.log(marks); // [2, 4, 8, 10, 15]
```

2. Lập trình Javascript

»» Array

- **forEach()**: duyệt qua các phần tử mảng.

```
let numbers = [36, 6, 9, 18, 22];
numbers.forEach(myFunction);
function myFunction(value, index, array) {
  console.log(index + ": " + value);
}
```

- **map()**: tạo ra mảng mới bằng cách thực hiện logic bất kỳ trên mỗi phần tử mảng ban đầu

```
let arr1 = [36, 6, 9, 18, 22];
let arr2 = arr1.map(myFunction);
function myFunction(value, index, array) {
  return value ** 2;
}
console.log(arr2); //[1296, 36, 81, 324, 484]
```


2. Lập trình Javascript

»»» Array

- **reduce()**: thực hiện logic tính toán trên các phần tử mảng và trả về giá trị duy nhất.

```
let numbers = [3, 2, 4, 1, 6, 8];
let sum = numbers.reduce(myFunction);
function myFunction(total, value, index, array) {
  return total + value;
}
console.log(sum); // 24
```

```
let numbers = [3, 2, 4, 1, 6, 8];
let sum = numbers.reduce(function(total, value) {
  return total + value;
}, 0);
console.log(sum); // 24
```

```
let numbers = [3, 2, 4, 1, 6, 8];
let sum = numbers.reduce((total, value) =>
  (total + value), 0);
console.log(sum); // 24
```

```
let numbers = [3, 2, 4, 1, 6, 8];
let sum = numbers.reduce(myFunction, 0);
function myFunction(total, value, index, array) {
  return total + value;
}
console.log(sum); // 24
```

*Giá trị khởi tạo cho
biến tích trữ*

- **reduceRight()**: tương tự như **reduce()** nhưng duyệt mảng từ phải qua trái.

2. Lập trình Javascript

»» Array

- **every()**: kiểm tra tất cả phần tử của mảng thỏa điều kiện chỉ định.

```
const numbers = [36, 3, 11, 16, 19, 22];
const over20 = numbers.every(myFunction);
function myFunction(value, index, array) {
  return value > 20;
}
console.log(over20); // false
```

- **some()**: kiểm tra **một** vài phần tử của mảng thỏa điều kiện chỉ định.

```
const numbers = [36, 3, 11, 16, 19, 22];
const over20 = numbers.some(myFunction);
function myFunction(value, index, array) {
  return value > 20;
}
console.log(over20); // true
```

2. Lập trình Javascript

»» Array

- **indexOf()**: trả về vị trí của phần tử đầu tiên tìm thấy trong mảng, nếu không tìm thấy trả về -1.

```
let numbers = [3, 2, 4, 3, 2, 8];  
let position = numbers.indexOf(2);  
console.log(position); // 1
```

- **lastIndexOf()**: trả về vị trí của phần tử cuối cùng tìm thấy trong mảng, nếu không tìm thấy trả về -1.

```
let numbers = [3, 2, 4, 3, 2, 8];  
let position = numbers.lastIndexOf(2);  
console.log(position); // 4
```

2. Lập trình Javascript

»» Array

- **find()**: trả về phần tử đầu tiên tìm thấy trong mảng, nếu không tìm thấy trả về *undefined*.

```
let beers = ["Heineken", "Tiger", "Sapporo"];
let beer = beers.find(function(value) {
  return value == "Tiger";
});
console.log(beer); // Tiger
```

- **filter()**: tạo ra mảng mới với **các** phần tử của mảng thỏa mãn điều kiện chỉ định.

```
const numbers = [36, 3, 11, 16, 19, 22];
const over20 = numbers.filter(myFunction);
function myFunction(value, index, array) {
  return value > 20;
}
console.log(over20); // [36, 22]
```

2. Lập trình Javascript

»» Array

- **includes()**: kiểm tra phần tử có tồn tại trong mảng, chuỗi con trong chuỗi gốc.

```
let beers = ["Heineken", "Tiger", "Sapporo"];  
console.log(beers.includes("Tiger")); // true
```

```
let s = "No Pain No Gain!"  
console.log(s.includes("No", 3)); //true
```

- **findIndex()**: trả về chỉ mục index của phần tử đầu tiên thỏa điều kiện chỉ định.

```
const numbers = [1, 3, 11, 16, 19, 22];  
const over15 = numbers.findIndex(myFunction);  
function myFunction(value, index, array) {  
    return value > 15;  
}  
console.log(over15); // 3
```